

An Adaptive Approach to Whole-Body Balance Control of Wheel-Bipedal Robot Ollie

Jingfan Zhang¹, Shuai Wang^{1,*}, Haitao Wang¹, Jie Lai¹, Zhenshan Bing²,
Yu Jiang³, Yu Zheng¹, *Senior Member, IEEE*, Zhengyou Zhang¹, *Fellow, IEEE*

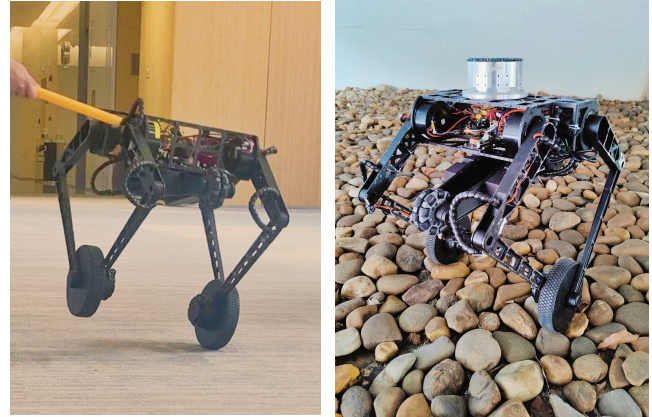
Abstract—The wheel-bipedal robot has the advantages of both wheeled robots and legged robots, but as a cost, it is more challenging to perform flexible movements in various surroundings while keeping it balanced. The inaccurate dynamics of the robot makes the balance problem even more intractable. To solve this problem, the robot Ollie is used as a testbed. The whole-body control (WBC) framework is adopted to enhance the dexterity of the robot with multiple degrees of freedom in the task space. Moreover, a learning-based adaptive technique is applied to assist the WBC such that the balance controller can be designed in the absence of the accurate dynamics. Physical experiments demonstrate that the robot can manage various actions, with the help of the combination of the WBC and the learning-based adaptive technique.

I. INTRODUCTION

A wheel-bipedal robot moves faster on the flat ground with high energy efficiency, and the legs of the robot lead to better adaptability and traversability on uneven terrains. However, there are two major challenges to control such a robot. Firstly, it is difficult to coordinate multiple joints to complete the diverse flexible movements. Secondly, as there are only two contact points between the robot and the ground, the robot is inherently unstable, and balance plays a pivotal role in the control of the robot. In this paper, we proposed a combined a learning-based adaptive technique and the whole-body control (WBC) method to overcome these limitations.

In order to manage diverse flexible movements in various scenarios, the WBC is adopted to enhance the dexterity of the robot. The WBC is a general framework that is extensively used in the robotic control to coordinate the multiple joints of the robot, see application to humanoid [1], wheel-bipedal [2] and quadrupedal robots [3]. The key idea of the WBC is to break down the nonlinear control problem of multiple degrees of freedom (DoF) robots to a linear controller in the task space and a feedforward optimization problem. The objective function of the optimization is the quantification of the performance and energy cost, considering constraints of the robot dynamics. Therefore, the WBC can tackle different tasks in both operational and joint space.

For an inherently unstable robot, the references of the joints' torques are required for the WBC such that the robot can be balanced. Traditionally, the reference signal is generated by the model-based balance controllers. For



(a) Overcome manual disturbances

(b) Drive on loose pebbles

Fig. 1. The wheel-bipedal robot Ollie. Many more maneuvers in action and experimental data are shown in the accompanying video.

example, for ETH Ascento [4], a template model is adopted to approximate the wheel-bipedal robot, and a model-based linear quadratic regulator balance controller is designed to generate the balance references to the WBC. For CMU Ballbot with arms [5], the PID controller is manually tuned for a simplified model in each direction. To avoid suboptimal performance caused by using approximate linearized models, H. Zhou *et al.* [6] model HIT-WLR-II as a high-order inverted pendulum and design the balance controller with nonlinear control methods [7], such as partial feedback linearization and the sliding mode control [8]. H. Chen *et al.* [9] propose a wheeled-spring-loaded inverted pendulum model to characterize the essential dynamics of Nezha for motion planning and use the WBC for accurate trajectories tracking. Other model-based nonlinear control methods are used for the balance control of the wheeled robots, see examples in [10]–[13]. The major limitation of the model-based method is that the balance performance highly depends on the accuracy of the dynamical model. Moreover, when the physical parameters change, such as height and load, the controller should be re-tuned manually, which deteriorates the automation of the robot. For example, when the passenger changes, the optimality of the balance control of Segway is hard to be preserved [14]. Hence, a learning-based balance control should be proposed which can be adaptive to model uncertainties and parameter changes.

In this paper, the adaptive dynamic programming (ADP) technique developed in [15], [16] is applied on Ollie to handle model uncertainties, which can learn a near optimal balance controller with the input-state data collected from the

¹J. Zhang, S. Wang, H. Wang, J. Lai, Y. Zheng, and Z. Zhang are with Tencent Robotics X, Tencent, Shenzhen, Guangdong, 518057, China.

²Z. Bing is with the Department of Informatics, Technical University of Munich, Munich, Germany.

³Y. Jiang is with ClearMotion, Inc., Billerica, MA, USA.

*Corresponding author, E-mail: shawnshwang@tencent.com

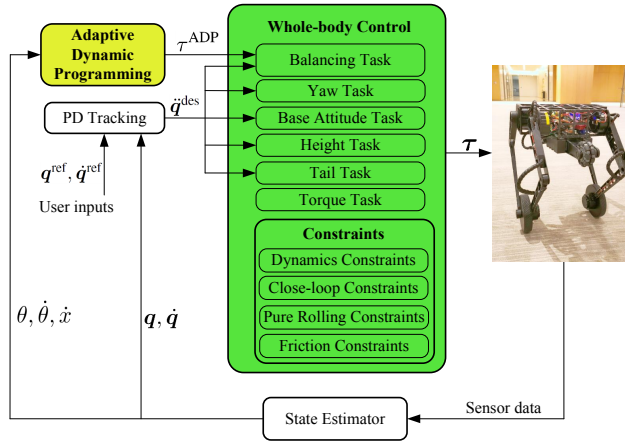


Fig. 2. Control framework of Ollie.

robot in the absence of the dynamic model. The ADP is a class of reinforcement learning methods, and the stability of the system in close-loop with an ADP-based controller can be proved theoretically by the Lyapunov theorem. Furthermore, the small data requirement in training makes it possible to directly collect data and to apply the learned control policy to the robot in real world, without considering transferring the learned policy from simulation to the real world. This forms an advantage compared with the conventional reinforcement learning application to real robots [17], [18]. ADP-based methods can be categorized into value iteration (VI) [16], [19] and policy iteration (PI) [15], [20]. From an initial stabilizing controller, PI can converge to the near optimal solution with the quadratic rate [21]. As a comparison, VI can learn the optimal controller without the initial stabilizing controller, but the rate of convergence of VI is slower than PI [16]. Although our previous work [22] has demonstrated effective results as the first application of the ADP on the balance control of the wheel-bipedal robot, there are still a list of limitations that deserve further investigation:

- In [22], training data was collected under an initial controller. This makes the methodology less applicable in reality, and does not take the advantage of the VI.
- Online adaptation is required when parameters of the robot change, but long iteration makes the VI offline only. How to update the optimal control policy online remains to be a problem.
- In [22], the robot used the wheels for balance and locked other joints, which restrains the dexterity of the robot that may need multiple joints to move simultaneously.

Contributions of this paper are listed as follows.

- We balance the inherently unstable robot Ollie without any initial controller, by collecting training data that represents the robot dynamics and applying VI algorithm (in Section III-B). The advantage of the VI algorithm is demonstrated.
- We use the ADP-based PI algorithm (in Section III-C) for the online adaptation of the control policy resulting from the VI during the change of working conditions.
- We combine the ADP-based balance controller with the

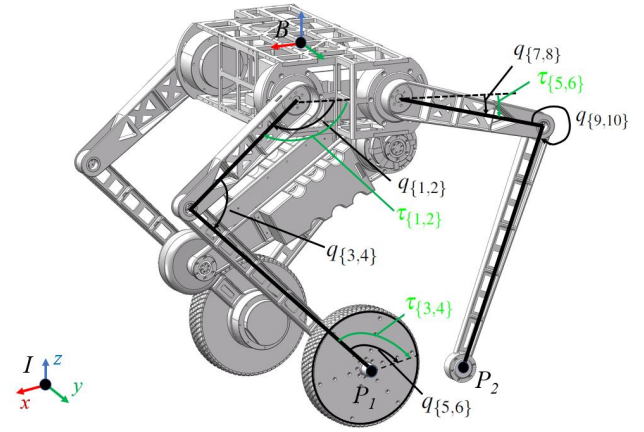


Fig. 3. Generalized coordinates of the robot with opened kinematic loops, with joints angles q_i and actuated motor torques τ_i labeled around joints. Double subscripts of $q_{\{.,.\}}$ and $\tau_{\{.,.\}}$ are marked considering indices of joints on the left and right legs, respectively. Only links and joints on the left leg are marked. Those on the right leg are symmetric. Two joints angles q_{11} , q_{12} and actuated torques τ_7 , τ_8 on the tail are omitted for clarification.

WBC (in Section II) to improve the dexterity of the robot, which shows a good performance in the real experiments (in Section IV-C).

II. ROBOTIC SYSTEM AND WHOLE-BODY CONTROL

A. Robotic system and control framework

The robotic system is introduced briefly. Totally eight actuators are used, including four on the hips, two on the tail and two on the wheels. The five-bar linkage mechanism is used so that each leg is actuated by two motors on the base, without actuators on knees. The robot weights 15 kg. The height of the robot varies between 0.33 m and 0.7 m.

To keep the robot balanced and to improve the moving ability with agile and flexible body poses, a control framework with the adaptive approach and the WBC is used (shown in Fig. 2). All motors are driven by torques. States of the robot are obtained by measurement sensors and the state estimation. Conventionally, for the balancing task in the WBC, references including joint angular accelerations or joint torques are generated by a simplified and linearized model and controllers [23]. To increase the adaptiveness and robustness of the balance controllers designed with respect to a specific simplified model, we calculate the balance references using the ADP-based control. Other task references, such as attitudes of the robot, are given by user inputs. The whole-body dynamics model and optimization formulation are given in the following subsections.

B. Dynamics Model

The generalized coordinates of the robot $\mathbf{q}$ include the pose of the floating base and n_j joint angles $\mathbf{q}_j = [q_1, q_2, \dots, q_{n_j}]^T$, where $n_j = 12$ for Ollie. The generalized velocities and accelerations are $\dot{\mathbf{q}}$ and $\ddot{\mathbf{q}}$. Torques of motors are represented using $\boldsymbol{\tau} = [\tau_1, \tau_2, \dots, \tau_{n_\tau}]^T$, where $n_\tau = 8$. Full dynamics model of the robot is

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) = \mathbf{S}^T \boldsymbol{\tau} + \mathbf{J}_f^T \mathbf{f} + \mathbf{J}_\lambda^T \boldsymbol{\lambda} \quad (1)$$

where $M(q) \in \mathbb{R}^{(6+n_j) \times (6+n_j)}$ is the mass matrix, $C(q, \dot{q}) \in \mathbb{R}^{6+n_j}$ consists of the gravity, centrifugal, and Coriolis terms, $S = [0_{n_r \times 6} \quad I_{n_r \times n_j}]$ is the matrix that selects the actuated joints from all joints, f is ground contact forces and $J_f \in \mathbb{R}^{3n_c \times (n_j+6)}$ is the concatenated contact Jacobian matrix, λ is loop closure forces and $J_\lambda \in \mathbb{R}^{3n_\lambda \times (n_j+6)}$ is the concatenated contact Jacobian matrix. Here n_c is the number of contact points between the wheels and the ground, and $n_\lambda = 2$ is the number of contact points between open-loop links considering the close-loop constraints of the five-bar mechanism, i.e. between points P_1 and P_2 in Fig. 3.

C. Optimization for the WBC

The WBC is can be formalized as an optimization:

$$\operatorname{argmin}_{(\tau, f, \lambda)} z = \|\ddot{q}^{\text{des}} - \ddot{q}\|_{W_q}^2 + \|\tau^{\text{des}} - \tau\|_{W_\tau}^2 + \|f\|_{W_f}^2 + \|\lambda\|_{W_\lambda}^2. \quad (2)$$

After solving this optimization, actuation torques τ are applied to drive the robot joints directly. Constraints of this optimization are listed as follows:

- (a) *Dynamics constraints*: This is described by (1).
- (b) *Close-loop constraints*:

$$J_\lambda \ddot{q} + \dot{J}_\lambda \dot{q} = 0 \quad (3)$$

where $J_\lambda^T = [J_{P_1, l}^T - J_{P_2, l}^T \quad J_{P_1, r}^T - J_{P_2, r}^T]^T$ and J_{P_1} and J_{P_2} are the Jacobian matrices of points P_1 and P_2 , respectively. The subscripts $J_{\cdot, l}$ and $J_{\cdot, r}$ denote the leg on the left and right, respectively. See [4] and [24] for similar techniques.

(c) *Pure rolling constraints*: Assuming that wheels are pure rolling and in contact with the ground, there exists no slipping and sliding in the wheel radial and axial directions,

$${}_B J_w^{(1,3)} \ddot{q} + {}_B \dot{J}_w^{(1,3)} \dot{q} = 0 \quad (4)$$

where ${}_B J_w^{(1,3)}$ is the x- and z-axis line of the Jacobian matrix of the wheel-ground contact point with respect to the base.

(d) *Friction constraints*: The friction cone is approximated with a friction pyramid. In the local frame of each contact force f_i , given a friction coefficient μ , the friction constraints are formulated as $|f_{i,x}| \leq \mu f_{i,z}$ and $|f_{i,y}| \leq \mu f_{i,z}$. The unilateral constraints can be formulated as $f_{i,z} > 0$.

D. Tasks

Among tasks of WBC, the balancing task references come from the ADP, and the other tasks are obtained from user input references. The desired acceleration of the floating base includes the base attitude and the height task, and is calculated by PD control with the reference poses and velocities. It is similar with $\ddot{q}_J$, except for the wheel joints:

$$\ddot{q}_i^{\text{des}} = \begin{cases} k_{q,p}(q_i^{\text{ref}} - q_i) + k_{q,d}(\dot{q}_i^{\text{ref}} - \dot{q}_i), & i \neq 5, 6, \\ 0, & i = 5, 6. \end{cases} \quad (5)$$

The desired torques for the wheels are obtained directly from the ADP and the others are set to zero:

$$\tau_i^{\text{des}} = \begin{cases} \tau^{\text{ADP}}, & i = 3, 4, \\ 0, & i \neq 3, 4. \end{cases} \quad (6)$$

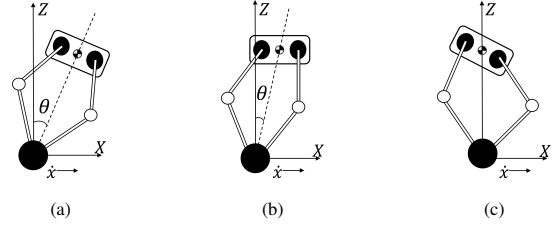


Fig. 4. Different poses of simplified models. (a) WIP model: the legs are symmetric. (b) WBC model: The robot is not at an equilibrium position although the floating base is horizontal. (c) WBC model: The robot is at an equilibrium position although the floating base is not horizontal.

III. BALANCE CONTROL ALGORITHM

A. Problem Statement

For a wheel-bipedal robot, it is balanced in the sagittal plane if the Center of Mass (CoM) of the upper body is right above the middle of two wheels. To realize this balancing behavior, the wheel-bipedal robot is often described by simplified models in the linear form

$$\dot{x} = Ax + B\tau. \quad (7)$$

In this paper, we regulate the CoM of the upper body and control the velocity of the robot by actuating the wheel motors. Henceforth, we define $x = [\theta, \dot{\theta}, \dot{x}]^T$, where θ is the pitch angle of the robot; $\dot{x}$ is the linear velocity of the robot; and τ is the sum of the torques of the two wheel motors. The graphical meaning of the states is shown in Fig. 4. Correspondingly, the matrices $A \in \mathbb{R}^{3 \times 3}$ and $B \in \mathbb{R}^{3 \times 1}$. However, due to the complex mechanical structure, it is hard to specify an accurate model for the balancing problem, and as a consequence, the values of A and B are unknown. Moreover, when the working condition of the robot changes (load, height, configuration, etc.), the model changes accordingly. For example, when the configuration of the robot is regulated as in Fig. 4a, the legs are symmetric, and the model can be simplified as a wheeled inverted pendulum (WIP) with a constant pendulum length [22]. However, due to the improvement of dexterity by WBC, the configuration of the robot may change to Fig. 4b and Fig. 4c, so the structure and the parameters of the proper model are both varying. If the balance controller is developed by a model-based method, after model varying, the controller should be re-tuned manually, which deteriorates the automation of the robot. Here we apply ADP to directly learn a near optimal controller online from the input-state data.

Due to the assumption of the linear model in (7), the optimal controller is in the form of $\tau(t) = -K^*x(t)$. According to the optimal control theory, $K^* = \frac{1}{R}B^T P^*$, where the positive-definite matrix P^* is the solution of the well-known Algebraic Riccati Equation (ARE)

$$A^T P^* + P^* A - \frac{1}{R} P^* B B^T P^* + Q = 0 \quad (8)$$

which corresponds to the optimization

$$\min_{\tau} J = \int_0^\infty x(t)^T Q x(t) + R \tau(t)^2 dt. \quad (9)$$

Algorithm 1: VI Algorithm

- 1: Choose an initial $\mathbf{P}$ and let $t_0 = 0$.
- 2: Apply an arbitrary $\tau_0(t)$ such that the robot keeps balance for a (short) period of time, and collect online data $\tau_0(t)$ and $\mathbf{x}(t)$.
- 3: Compute the matrices in (11) with a sufficiently large integer r .
- 4: Ensure the rank condition below holds.

$$\text{rank} \begin{bmatrix} \Sigma_{xx} & \Sigma_{x\tau} \end{bmatrix} = 9.$$
- 5: **While** $\| \frac{d\mathbf{P}(s)}{ds} \| < \epsilon$ with some small ϵ
- 6: Solve $\mathbf{H}(s)$ and $\mathbf{K}(s)$ by the equation

$$\Omega \begin{bmatrix} \text{vec}(\mathbf{H}(s)) \\ \text{vec}(\mathbf{K}(s)) \end{bmatrix} = \Delta_{xx} \text{vec}(\mathbf{P}(s))$$

where $\Omega = [\Sigma_{xx}, 2\Sigma_{x\tau}(\mathbf{I}_n R)]$, and update $\mathbf{P}(s)$ by (10) with an appropriate step length.
- 7: **End while**
- 8: Use $\tau = -\mathbf{K}(s)\mathbf{x}$ as the optimal control policy.

In the cost function, the positive semi-definite matrix $\mathbf{Q} \in \mathbb{R}^{3 \times 3}$ and the positive scalar R are the designed weights.

Clearly, to solve (8), $\mathbf{A}$ and $\mathbf{B}$ should be known and being constant, but in practice, the precise values are hard to get. To deal with this problem, two data-driven approaches based on ADP are applied to approximate the solution of (8) iteratively in the next two parts. Firstly, VI is applied to learn an near optimal controller from the scratch, and then PI is applied to update the controller from VI online such that the robot can adapt to rapid dynamic changes.

B. Value iteration ADP

Because of the inherent instability of the wheel-bipedal robot and the lack of initial balancing controllers, we apply VI algorithm to learn a stabilizing controller offline. The main idea of the VI algorithm is based on the iteration of the matrix $\mathbf{P}$ by the differential Riccati equation

$$-\frac{d\mathbf{P}(s)}{ds} = \mathbf{A}^T \mathbf{P}(s) + \mathbf{P}(s) \mathbf{A} - \frac{1}{R} \mathbf{P}(s) \mathbf{B} \mathbf{B}^T \mathbf{P}(s) + \mathbf{Q} \quad (10)$$

with some initial condition on $\mathbf{P}$. Then, by considering $\mathbf{H}(s) = \mathbf{A}^T \mathbf{P}(s) + \mathbf{P}(s) \mathbf{A}$ and $\mathbf{K}(s) = \frac{1}{R} \mathbf{B}^T \mathbf{P}(s)$, the dependency on $\mathbf{A}$ and $\mathbf{B}$ can be avoided if $\mathbf{H}(s)$ and $\mathbf{K}(s)$ are solved.

Based on this idea, the VI algorithm used in this paper is listed in Algorithm 1. Here, the operator $\otimes$ denotes the Kronecker product; $\text{vec}(\cdot)$ denotes the vectorization; and the following matrices $\Delta_{xx} \in \mathbb{R}^{(r+1) \times 9}$, $\Sigma_{xx} \in \mathbb{R}^{(r+1) \times 9}$ and $\Sigma_{x\tau} \in \mathbb{R}^{(r+1) \times 3}$ are defined.

$$\begin{aligned} \Delta_{xx} &= [\mathbf{x} \otimes \mathbf{x}|_{t_0}^{t_0+\delta t}, \mathbf{x} \otimes \mathbf{x}|_{t_1}^{t_1+\delta t}, \dots, \mathbf{x} \otimes \mathbf{x}|_{t_r}^{t_r+\delta t}]^T \\ \Sigma_{xx} &= \left[\int_{t_0}^{t_0+\delta t} \mathbf{x} \otimes \mathbf{x} dt, \int_{t_1}^{t_1+\delta t} \mathbf{x} \otimes \mathbf{x} dt, \dots, \int_{t_r}^{t_r+\delta t} \mathbf{x} \otimes \mathbf{x} dt \right]^T \\ \Sigma_{x\tau} &= \left[\int_{t_0}^{t_0+\delta t} \mathbf{x} \tau_0 dt, \int_{t_1}^{t_1+\delta t} \mathbf{x} \tau_0 dt, \dots, \int_{t_r}^{t_r+\delta t} \mathbf{x} \tau_0 dt \right]^T \end{aligned} \quad (11)$$

where $0 \leq t_0 < t_1 < \dots < t_r$, and $\delta t \leq t_{l+1} - t_l$ ($l = 0, 1, \dots, r-1$).

In Algorithm 1, it is worth highlighting that the model information ($\mathbf{A}, \mathbf{B}$) are not used. Moreover, the control input τ_0 can be arbitrarily chosen. Therefore, by the VI algorithm, the balance of the robot can be trained from the initial unstable behavior without any systematic design on the input signal. The detailed derivation of Algorithm 1 is provided in [22] and is not repeated here.

Algorithm 2: PI Algorithm

- 1: Choose an stabilizing initial gain matrix $\mathbf{K}_0$ and let $t_0 = 0$.
- 2: Apply $\tau_0(t) = -\mathbf{K}_0 \mathbf{x}(t) + \beta(t)$ with a noise $\beta(t)$, and collect online data $\tau_0(t)$ and $\mathbf{x}(t)$.
- 3: Compute the matrices in (11) with a sufficiently large integer r .
- 4: Ensure the rank condition below holds.

$$\text{rank} \begin{bmatrix} \Sigma_{xx} & \Sigma_{x\tau} \end{bmatrix} = 9.$$
- 5: **While** $\|\mathbf{P}_{k+1} - \mathbf{P}_k\| < \epsilon$ with some small ϵ
- 6: Solve $\mathbf{P}_k$ and $\mathbf{K}_{k+1}$ by the equation

$$\tilde{\Omega}_k \begin{bmatrix} \text{vec}(\mathbf{P}_k) \\ \text{vec}(\mathbf{K}_{k+1}) \end{bmatrix} = \Xi_k$$

where $\tilde{\Omega}_k = [\Delta_{xx}, -2\Sigma_{xx}(\mathbf{I}_n \otimes \mathbf{K}_k^T R) - 2\Sigma_{x\tau}(\mathbf{I}_n R)]$, and $\Xi_k = -\Sigma_{xx} \text{vec}(\mathbf{Q} + \mathbf{K}_k^T R \mathbf{K}_k)$.
- 7: **End while**
- 8: Use $\tau = -\mathbf{K}_k \mathbf{x}$ as the optimal control policy.

However, a cost for the relaxation on the input is the slow convergence, making the rate sublinear [16]. In consequence, the VI algorithm is difficult to be implemented online. To realize the online training, the following PI algorithm is introduced.

C. Policy iteration ADP

Different from the VI, the PI algorithm is based on the Lyapunov equation

$$-\mathbf{K}_k^T R \mathbf{K}_k - \mathbf{Q} = \mathbf{A}_k^T \mathbf{P}_k + \mathbf{P}_k \mathbf{A}_k \quad (12)$$

and the gain iteration

$$R \mathbf{K}_{k+1} = \mathbf{B}^T \mathbf{P}_k. \quad (13)$$

Then, during the derivation of the PI algorithm, the term $\mathbf{A}^T \mathbf{P} + \mathbf{P} \mathbf{A}$ can be replaced using the left hand side of (12), and the term $\mathbf{B}^T \mathbf{P}$ can be replaced using the left hand side of (13). Based on this idea, the PI algorithm used in this paper is listed in Algorithm 2.

Similar to the VI algorithm, the PI algorithm is also independent with $\mathbf{A}$ and $\mathbf{B}$. Moreover, $\mathbf{P}_k$ and $\mathbf{K}_{k+1}$ are solved simultaneously. Importantly, (11) collects all the online measures before the iteration of the the control policy from $\mathbf{K}_k$ to $\mathbf{K}_{k+1}$. Therefore, the collected data in (11) can be used repeatedly to update $\mathbf{K}_k$ for $k = 0, 1, \dots$, and the updating process can be either online or offline. For this reason, this strategy is called off-policy iteration [15].

The difference from the VI algorithm is the extra requirement on the structure and the stability of the initial input τ_0 . Nevertheless, due to this priori, the convergence rate of the PI is quadratic [21]. The faster convergence than the VI allows the online implementation. The detailed derivation of Algorithm 2 is provided in [15] and is not repeated here.

Remark 1: In Algorithm 2, the artificial noise β , called the exploration noise, is commonly used in learning and system identification fields [25]. As the robot is balanced, it always operates around its equilibrium. Hence, the noise can trigger various system behaviors to avoid the repetition of the collected data.

Remark 2: Due to the unmeasurable system noise, the collected data may deviate from the ideal form (7). With the noisy data, the robustness of the PI algorithm guarantees the convergence to a small neighbor of the optimal policy if the noise is bounded and relatively small [20].

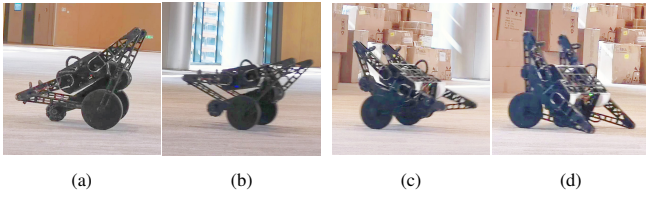


Fig. 5. Photos during the training for an initial stable controller.

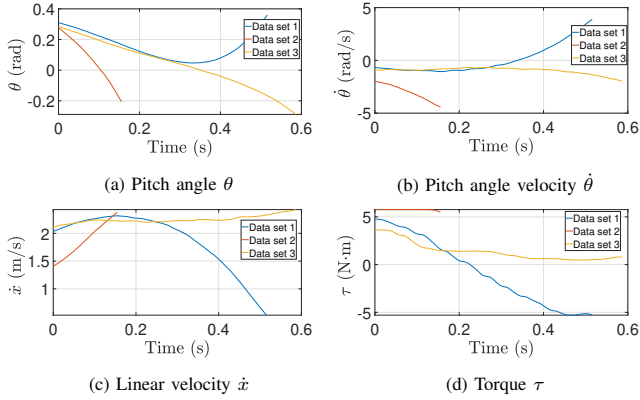


Fig. 6. Training data for the VI algorithm.

IV. EXPERIMENTAL RESULTS

In this section, we apply the adaptive approach to train the balance controller in the WBC framework. First, we train an initial controller offline from the unstable behavior of the robot using Algorithm 1. Second, we update an adaptive optimal controller online when the working condition of the robot changes using Algorithm 2. Finally, a few experiments demonstrate the dexterity of the robot by using the WBC with the resultant balance controller. Although ADP-based approaches are also used in other robotic systems in recent studies, such as manipulations [26], [27] and semi-autonomous vehicles [28], the hardware implementations for unstable mobile robots are less common.

A. Training an initial stable controller

1) *Training*: Ollie is an inherently unstable robot, so we directly train an initial stabilizing controller from its unstable data. In this case, Algorithm 1 is appropriate because the input can be arbitrary.

In this training experiment, the hip motors are locked to make the robot at the minimum height 0.33 m as shown in Fig. 5, and the wheel motor torques are directly given by the remote controller manually. Notice that the WBC is hard to be applied at this moment because the balancing task in the WBC framework has not been realized. In the experiment, with the input torque of the wheel motors increasing, the robot starts from the initial condition (Fig. 5a), then moves across its equilibrium (Fig. 5b, 5c), and finally falls down on the other side (Fig. 5d). The process covers the speed up and the slow down, and the pitch angle also increases and decreases. This informative data are more likely to cover all dynamics. The similar process is repeated for three times, with data plotted in Fig. 6, where Fig. 6d is the total torque of the two wheel motors. Particularly, as the system

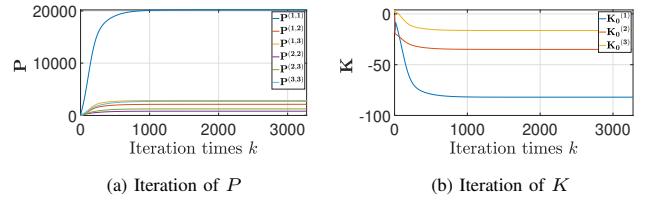


Fig. 7. Convergence of the VI algorithm.

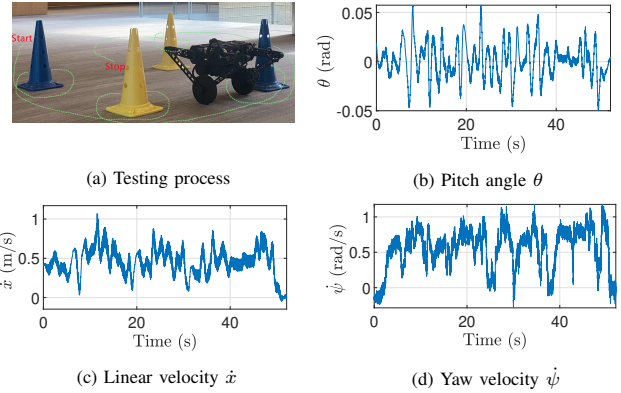


Fig. 8. Testing photo and results of the VI result.

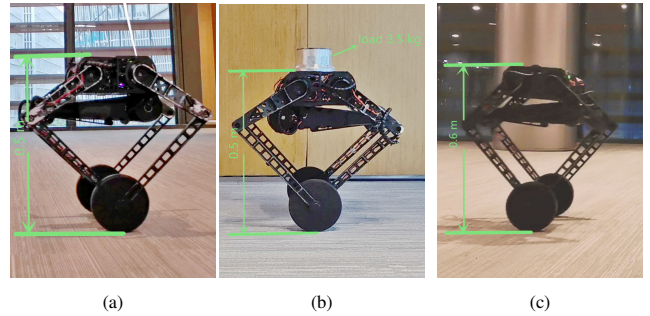


Fig. 9. Photos during the training for updating optimal controllers. (a) height = 0.5 m. (b) height = 0.5 m with a load = 3.5 kg. (c) height = 0.6 m.

is assumed to be linear, the data close to the linear region, i.e. $-0.35 \text{ rad} < \theta < 0.35 \text{ rad}$, are used.

It should be highlighted that, during these training processes, the robot is unstable. This is different from our previous work [22] where a stable initial controller is used to train the optimal controller by the VI algorithm. In addition, the time duration of the three data sets in Fig. 6 are 0.515 s, 0.155 s, 0.586 s respectively, and 1.256 s in total. This short data can be easily collected by any nonprofessional by a few attempts. In contrast, data for more than 12 s are used in [22], and such a long valid data is impractical when the robot is unstable. Therefore, the training process in this paper addresses the efficiency of Algorithm 1. Furthermore, as the VI algorithm can be conducted offline, once the collected data satisfies the rank condition in Algorithm 1, it is not a challenge to tune the parameters of the VI to make it converge. After a converging result appears, the trade-off between the fast response and the little oscillation can be adjusted by tuning the weights in the LQR (9).

Finally, using Algorithm 1 with the data in Fig. 6, and setting $R = 20$, $Q = \text{diag}(20000, 8000, 3000)$, the iteration

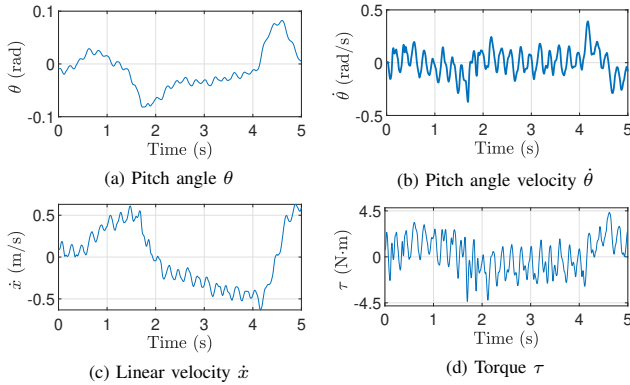


Fig. 10. PI Training data when height = 0.5 m.

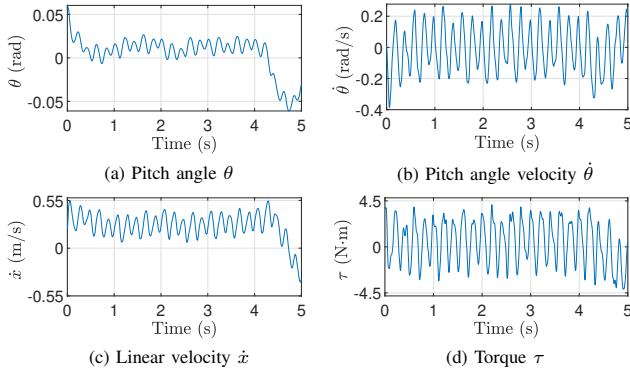


Fig. 11. PI Training data when height = 0.5 m with the load.

of $\mathbf{P}$ and the control gain $\mathbf{K}$ is demonstrated in Fig. 7. The iteration takes 3275 steps to reach the tolerance $\epsilon = 0.005$. The resultant gain is $\mathbf{K}_0 = [-81.99 \quad -34.96 \quad -16.38]$, which will be used as the initial policy in the following experiments. The VI algorithm is offline, so the complicity in data processing and its slow convergence are acceptable.

2) *Testing*: According to the VI algorithm, its resultant controller should be stable, so it is safe to use the WBC on the robot. According to the base attitude and the relative velocity between the base and the wheels, the state $\mathbf{x} = [\theta, \dot{\theta}, \dot{x}]^T$ can be calculated. The resultant τ^{ADP} in (6) is used as reference torques of both wheels in the WBC. In the testing process, the robot rotates around every pile anticlockwise as shown in Fig. 8a, where the reference linear velocity and the reference yaw velocity is given by the remote controller.

The experimental results are shown in Fig. 8. The robot keeps balanced as the pitch angle θ is in the small range $[-0.0499 \text{ rad}, 0.0574 \text{ rad}]$. This result implies the strong regulation of the control policy on the pitch angle, which is determined by the weights in the $\mathbf{Q}$ matrix. In addition, the positiveness of the linear velocity $\dot{x}$ and the yaw velocity $\dot{\psi}$ implies that the robot is moving forward and rotating anticlockwise. It has to be recognized that small oscillations exist in $\dot{x}$ and $\dot{\psi}$, but it is less significant.

In conclusion, by the VI algorithm, it is possible to achieve a stable controller using small data from the unstable behavior of the robot.

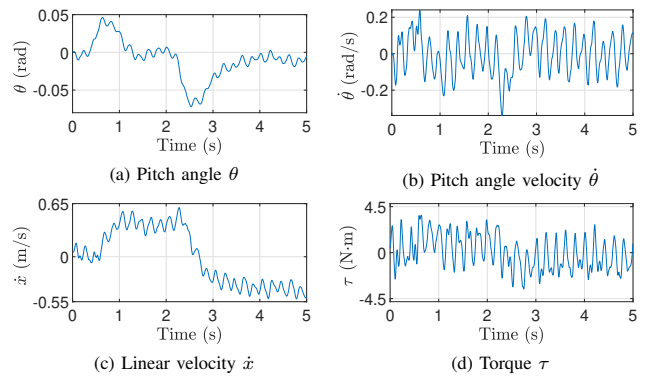


Fig. 12. PI Training data when height = 0.6 m.

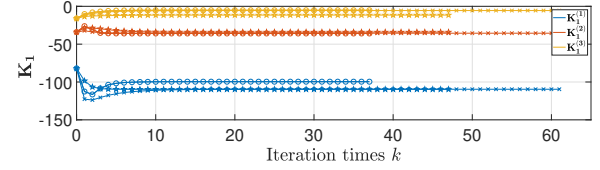


Fig. 13. Convergence of the PI algorithm for the three experiments: circle-marked for $\mathbf{K}_{1,a}$ when height = 0.5 m; cross-marked for $\mathbf{K}_{1,b}$ when height = 0.5 m with the load; pentagram-marked for $\mathbf{K}_{1,c}$ when height = 0.6 m.

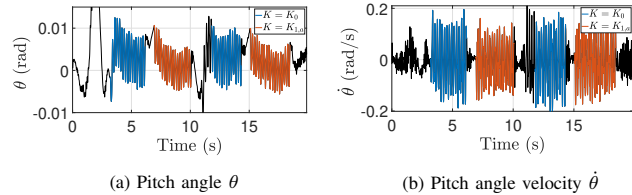


Fig. 14. Testing results when using different controllers.

B. Updating optimal controllers

1) *Training*: After the VI training in the last section, a stable controller is available for the minimum height of the robot. However, when the working condition changes, the intrinsic dynamics also changes, and the optimal control policy should be updated online. We consider three working conditions: height = 0.5 m, height = 0.5 m with a 3.5 kg load, and height = 0.6 m, as shown in Fig. 9. Here, we use the PI algorithm in Algorithm 2 online to update the optimal controller adaptively. This online adaptation is not considered in our previous work [22]. Notice that, the information of the height and the load are not used in the training process.

In the training experiments, we use the initial gain $\mathbf{K}_0$ from the VI training, and the noise $\beta(t) = \sin(10\pi t) + 0.4\cos(6\pi t)$. Experimental data of 5 s are collected for each working condition as shown in Fig. 10, 11 and 12 respectively. Then, by the PI Algorithm 2 with the same $\mathbf{Q}$ and $\mathbf{R}$, the optimal gains for the three cases are

$$\begin{aligned} \mathbf{K}_{1,a} &= [-99.58 \quad -35.87 \quad -5.18], \\ \mathbf{K}_{1,b} &= [-109.57 \quad -35.52 \quad -5.58], \\ \mathbf{K}_{1,c} &= [-109.64 \quad -34.08 \quad -11.58], \end{aligned} \quad (14)$$

respectively, for which the iteration take 37 steps, 61 steps and 47 steps to reach the the tolerance $\epsilon = 10^{-5}$ (Fig. 13).

It is clear that the PI algorithm converges much faster than the VI, which is the advantage of PI for online training.

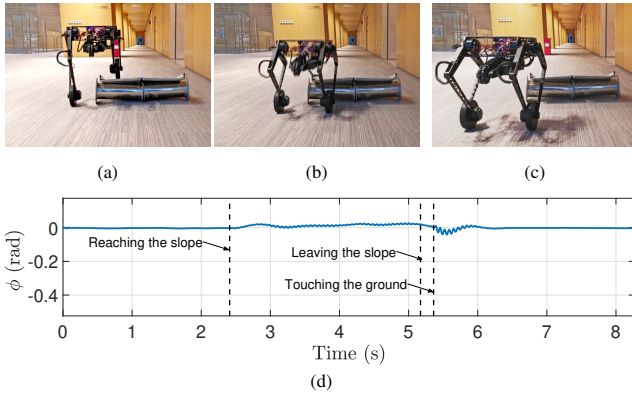


Fig. 15. Photos and results of Experiment 2 by the WBC. (a) Before leaving the slope (b) Touching the ground (c) After touching the ground (d) Roll angle during the experiment. For detail please see our accompanying video.

2) *Testing*: To show the adaptive optimization during the PI process, the performance of the resultant control gain $K_{1,a}$ is compared with the initial gain K_0 when the robot height is 0.5 m. In the experiment, the same sinusoidal noise is added to the input torque of wheel motors to simulate the disturbance acted on the wheels. As illustrated by Fig. 14, both controllers are robust to the noise and the control performances are similar. However, the updated gain $K_{1,a}$ is slightly stronger in the regulation of θ as implied by the smaller amplitude of with $K = K_{1,a}$ in Fig. 14b.

In addition, we test the stability of the controller $K_{1,b}$ trained by the PI when the load is added. The robot with the load can drive on the uneven loose pebbles stably as shown in Fig. 1b. This result shows the nice robustness of the balance controller and the strong off-load capability of the robot under the WBC framework. To further demonstrate the dexterity brought by the WBC, a few tests are conducted in the following part.

C. Dexterity by WBC

With the help of the balancing algorithm by VI and PI, the dexterity of the multiple-DoF robot could be fully realized under the WBC framework. In the following testing scenarios, we set the height = 0.5 m, and use the controller gain $K_{1,a}$ for the balancing task in the WBC framework.

Experiment 1 (Fig. 1a). We add manual disturbances by a stick to the robot in all directions. The robot can overcome the disturbance especially in the x -direction due to the strong regulation by the balance controller. When the disturbance is added in the y -direction, the torque controlled hip motors are compliant to the external force. However, when the disturbance is aggressive, one wheel would leave the ground. It must be recognized that the nonholonomic constraints in WBC address that the wheels are in contact with the ground, but the robustness of the system allows the wheel to leave the ground for a short time.

Experiment 2 (Fig. 15-Fig. 16). In Experiment 1, the robot can leave the ground for a while, so in this experiment, the robot moves on a slope of 11° with the left wheel, while the right wheel keeps on the ground, as shown in Fig. 15. In Fig. 15a, the robot is balanced on the slope, and the base

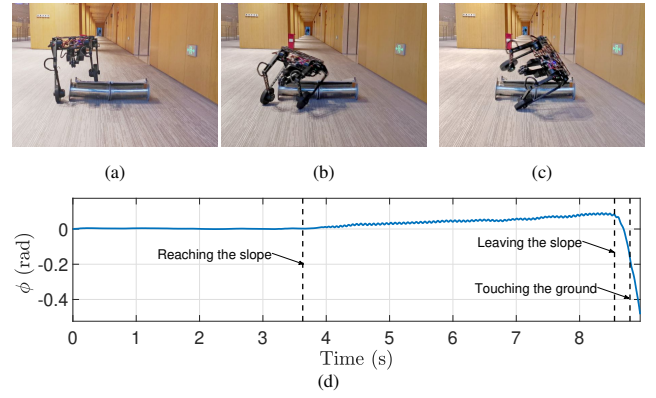


Fig. 16. Photos and results of Experiment 2 by [22]. (a) Before leaving the slope (b) Touching the ground (c) After touching the ground (d) Roll angle during the experiment. For detail please see our accompanying video.

keeps horizontal when the robot is moving on the slope. In Fig. 15b - 15c, the robot is also balanced after falling onto the ground. During the whole experiment, the roll angle of the base is between -0.04 rad and 0.026 rad (Fig. 16d). This result illustrates the fast response of the WBC in the base pose tracking. A major difference to our previous work [22] is compared in Fig. 16, where the hip motors are position controlled. Although the robot responds properly to the slow change of the height of the slope (Fig. 16a), but the response is over slow for the sudden change of the height at the end of the slope (Fig. 16b), and the robot finally falls over (Fig. 16c). Particularly, the roll angle of the base reaches 0.09° on the slope (Fig. 16d), which is 3.5 times of the WBC case. The main reason for the difference compared to the WBC performance is that the hip motor is position controlled, so the response of the motor is slower. In this experiment, the distinct improvement on the dexterity due to the application of the WBC is proved, which is a nontrivial contribution compared to [22].

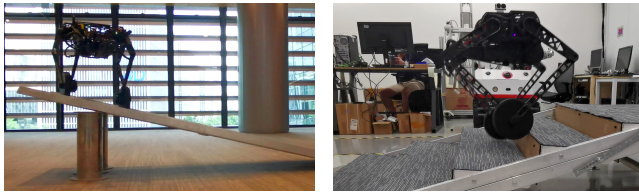
Experiment 3 (Fig. 17a). In this experiment, the robot moves on the slope and makes a U-turn. As shown in Fig. 17a, the hip motors react smoothly to tune lengths of the two legs to keep the base pose on the slope, and the balance controller guarantees the stability of the robot. This challenging task is impossible in [22]. This result shows the ability of the robot to move freely on inclined grounds.

Experiment 4 (Fig. 17b). In this experiment, the robot can drive downstairs continuously (Fig. 17b), which indicates the fast response of the balance controller and the dexterity by the WBC. This result shows the off-road capability of the robot under the WBC framework.

In brief, the experiments above demonstrate the high dexterity of the robot by the WBC with the help of the balance control. This enables the off-road capability of the robot, and make potential wider outdoor functions possible.

V. CONCLUSIONS

The wheel-bipedal robot Ollie is used as a testbed for validating the adaptive WBC. Thanks to the small data requirement in the ADP, it makes the training data collection on the physical robot possible, so that we need not to consider



(a) Make a U-turn on a slope

(b) Drive downstairs

Fig. 17. Photos of Experiment 3 and 4.

simulation and the Sim-to-Real gap. Several limitations in our previous publication are solved in this paper. Firstly, convincing experimental results are given to demonstrate that the balance controller of the robot can be obtained using 1.256 s training data without an initial controller. To the best of the authors' knowledge, this is the first experiment that ADP-based balance control is performed on a real robot starting to learn from nothing. In addition, to deal with the working condition change during operation, an adaptive strategy is obtained via online PI ADP. Furthermore, the dexterity of the robot is realized by embedding the ADP-based balance control in the WBC. It shows the adaptive approach is suitable in more general control framework, and this control technique makes the outdoor applications of wheel-bipedal robots possible.

A remaining limitation of this research is disturbances are not considered in training. For example, when bearing the load, the CoM of the floating base deviates from the geometric center; when standing on the slope, the real equilibrium changes. In consequence, the robot is balanced but cannot stand still with the load or on the slope. This limitation deserves to be studied.

ACKNOWLEDGMENT

The authors acknowledge L. Cui, Prof. Z.-P. Jiang for their useful discussions, and D. Zhang, K. Chen, J. Wang, and Z. Li for efforts in the system integration and experiments.

REFERENCES

- [1] A. Escande, N. Mansard, and P.-B. Wieber, "Hierarchical quadratic programming: Fast online humanoid-robot motion generation," *The International Journal of Robotics Research*, vol. 33, no. 7, pp. 1006–1028, 2014.
- [2] S. Xin and S. Vijayakumar, "Online dynamic motion planning and control for wheeled biped robots," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2020, pp. 3892–3899.
- [3] C. D. Bellicoso, F. Jenelten, P. Fankhauser, C. Gehring, J. Hwangbo, and M. Hutter, "Dynamic locomotion and whole-body control for quadrupedal robots," in *Proceedings of 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2017, pp. 3359–3365.
- [4] V. Klemm, A. Morra, L. Gulich, D. Mannhart, D. Rohr, M. Kamel, Y. de Viragh, and R. Siegwart, "LQR-assisted whole-body control of a wheeled bipedal robot with kinematic loops," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3745–3752, 2020.
- [5] U. Nagarajan, B. Kim, and R. Hollis, "Planning in high-dimensional shape space for a single-wheeled balancing mobile robot with arms," in *2012 IEEE International Conference on Robotics and Automation*, 2012, pp. 130–135.
- [6] X. Li, S. Zhang, H. Zhou, H. Feng, and Y. Fu, "Locomotion adaption for hydraulic humanoid wheel-legged robots over rough terrains," *International Journal of Humanoid Robotics*, vol. 18, no. 1, pp. 1–22, 2021.
- [7] H. Zhou, H. Yu, X. Li, H. Feng, S. Zhang, and Y. Fu, "Configuration transformation of the wheel-legged robot using inverse dynamics control," in *Proceedings of 2021 IEEE International Conference on Robotics and Automation*, 2021, pp. 3091–3096.
- [8] J. Huang, Z.-H. Guan, T. Matsuno, T. Fukuda, and K. Sekiyama, "Sliding-mode velocity control of mobile-wheeled inverted-pendulum systems," *IEEE Transactions on Robotics*, vol. 26, no. 4, pp. 750–758, 2010.
- [9] H. Chen, B. Wang, Z. Hong, C. Shen, P. M. Wensing, and W. Zhang, "Underactuated motion planning and control for jumping with wheeled-bipedal robots," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 747–754, 2021.
- [10] S. Wang, L. Cui, J. Zhang, J. Lai, D. Zhang, K. Chen, Y. Zheng, Z. Zhang, and Z.-P. Jiang, "Balance control of a novel wheel-legged robot: Design and experiment," in *Proceedings of 2021 International Conference on Robotics and Automation*, 2021, pp. 6782–6788.
- [11] L. Cui, S. Wang, J. Lai, X. Chen, S. Yang, Z. Zhang, and Z.-P. Jiang, "Nonlinear balance control of an unmanned bicycle: Design and experiments," in *Proceedings of 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2020, pp. 7279–7284.
- [12] S. Wang, L. Cui, J. Lai, S. Yang, X. Chen, Y. Zheng, Z. Zhang, and Z.-P. Jiang, "Gain scheduled controller design for balancing an autonomous bicycle," in *Proceedings of 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2020, pp. 7595–7600.
- [13] L. Cui, S. Wang, Z. Zhang, and Z.-P. Jiang, "Asymptotic trajectory tracking of autonomous bicycles via backstepping and optimal control," *IEEE Control Systems Letters*, vol. 6, pp. 1292–1297, 2021.
- [14] R. Babazadeh, A. G. Khiabani, and H. Azmi, "Optimal control of segway personal transporter," in *4th International Conference on Control, Instrumentation, and Automation (ICCIA)*, 2016, pp. 18–22.
- [15] Y. Jiang and Z.-P. Jiang, *Robust Adaptive Dynamic Programming*. NJ, USA: Wiley-IEEE Press, 2017.
- [16] T. Bian and Z.-P. Jiang, "Value iteration and adaptive dynamic programming for data-driven adaptive optimal control design," *Automatica*, vol. 71, pp. 348–360, 2016.
- [17] Y. Zhou, J. Lin, S. Wang, and C. Zhang, "Learning ball-balancing robot through deep reinforcement learning," in *Proceedings of International Conference on Computer, Control & Robotics*, 2021, pp. 1–8.
- [18] C. Yang, K. Yuan, Q. Zhu, W. Yu, and Z. Li, "Multi-expert learning of adaptive legged locomotion," *Science Robotics*, vol. 5, no. 49, 2020.
- [19] T. Bian and Z.-P. Jiang, "Reinforcement learning and adaptive optimal control for continuous-time nonlinear systems: A value iteration approach," *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–10, 2021.
- [20] B. Pang, T. Bian, and Z.-P. Jiang, "Robust policy iteration for continuous-time linear quadratic regulation," *IEEE Transactions on Automatic Control*, vol. 67, no. 1, pp. 504–511, 2022.
- [21] D. Kleinman, "On an iterative technique for Riccati equation computations," *IEEE Transactions on Automatic Control*, vol. 13, no. 1, pp. 114–115, 1968.
- [22] L. Cui, S. Wang, J. Zhang, D. Zhang, J. Lai, Y. Zheng, Z. Zhang, and Z.-P. Jiang, "Learning-based balance control of wheel-legged robots," *IEEE Robotics and Automation Letters*, 2021.
- [23] V. Klemm, A. Morra, C. Salzmann, F. Tschopp, K. Bodie, L. Gulich, N. Küng, D. Mannhart, C. Pfister, M. Vierneisel, F. Weber, R. Deuber, and R. Siegwart, "Ascento: A two-wheeled jumping robot," in *Proceedings of 2019 International Conference on Robotics and Automation*, 2019, pp. 7515–7521.
- [24] R. Featherstone and D. Orin, "Robot dynamics: equations and algorithms," in *Proceedings of 2000 IEEE International Conference on Robotics and Automation*, vol. 1, 2000, pp. 826–834.
- [25] R. Pintelon and J. Schoukens, *System identification: a frequency domain approach*. John Wiley & Sons, 2012.
- [26] D. H. Hong Zhan and C. Yang, "Adaptive dynamic programming enhanced admittance control for robots with environment interaction and actuator saturation," *International Journal of Intelligent Robotics and Applications*, vol. 5, pp. 89–100, 2021.
- [27] L. Kong, S. Zhang, and X. Yu, "Approximate optimal control for an uncertain robot based on adaptive dynamic programming," *Neurocomputing*, vol. 423, pp. 308–317, 2021.
- [28] M. Huang, W. Gao, Y. Wang, and Z.-P. Jiang, "Data-driven shared steering control of semi-autonomous vehicles," *IEEE Transactions on Human-Machine Systems*, vol. 49, no. 4, pp. 350–361, 2019.